GRANDPIED · IT DEPARTMENT

SpamJam

Detecting spam without losing a client email

Final project report, Machine Learning × Advanced Python · Victoria Alajemba and Sheila Kana · Teacher: Souheil Hanoune · November 2026

1. Executive summary

GrandPied, a (fictional) marketing company, runs on email, and its staff receive more and more spam. SpamJam is a proof of concept of a machine-learning spam filter for its IT department. A blocked client email can cost a contract, while a missed spam is only a nuisance, so we treat false positives (legitimate email flagged as spam) as the more costly error and choose our metric and decision threshold accordingly.

- **Model.** Three tuned models (logistic regression, k-NN, linear SVM) reach a cross-validated average precision of about 0.954, against 0.399 for a baseline that never flags spam. They are statistically tied, so we keep the simplest and most interpretable: logistic regression.
- **Test set.** With a decision threshold of 0.782, chosen on out-of-fold predictions, the model reaches a precision of 0.950 and a recall of 0.795 on the untouched test set: 14 of 506 legitimate emails are flagged, and 69 of 336 spam get through.
- **Main limitation.** All legitimate emails in the dataset come from one person, so some of the model's strongest signals (the word “george”) would not exist at GrandPied. A second version trained without these words keeps the same precision (0.952) but catches less spam (recall 0.711). The model is also English only, and GrandPied's own marketing emails look like spam to it, so it must be retrained on GrandPied's emails before real use.

2. Problem and success criteria

Task. Supervised binary classification: predict `is_spam` (1 = spam, 0 = ham, a legitimate email) from 57 numeric features describing an email.

Cost of errors. A false positive sends a client email to the spam folder, where it may never be read. A false negative lets a spam through, which staff can delete. GrandPied accepts more of the second to avoid the first, so we favour precision on the spam class.

Metric. We select models with average precision (the area under the precision-recall curve). It summarises precision over every threshold, focuses on the spam class, and is not inflated by the larger ham class, unlike accuracy. Once the model is chosen, we set the decision threshold so that precision on spam reaches 0.95. We also report recall, F1, ROC-AUC and the confusion matrix.

Decision rule. One option replaces a simpler one only if its cross-validated score is higher by more than one standard error; otherwise we keep the simpler one.

3. Data

We use the Spambase dataset from the UCI Machine Learning Repository (Hopkins, Reeber, Forman and Suermondt, 1999; CC BY 4.0). Every column is described in the project's data dictionary (`docs/data_dictionary.md`).

Item	Value
Emails	4,601 (39.4 % spam), no missing values
Features	57 numeric: 48 word frequencies, 6 character frequencies, 3 capital-letter run lengths
Cleaning	391 exact duplicates removed, leaving 4,210 emails
Split	Stratified 80/20, seed 42: 3,368 training and 842 test emails (39.9 % spam in each), saved so every notebook and command uses the same split

Table 1. The dataset after cleaning.

Exploration (on the training set only) led to three findings. The capital-letter run lengths are very skewed (skewness 9.7 to 32.1), so we log-transform them. The features `857` and `415` are almost identical (correlation 0.994). And some words describe the dataset's donor (George, at HP Labs) rather than spam: `george`, `hp`, `hpl`, `650`, `857`, `415`, `labs`, `telnet`, `technology`. We return to them in section 5.3.

4. Method

Preprocessing pipeline. All steps run inside one scikit-learn `Pipeline` with a `ColumnTransformer`, so in cross-validation each fold learns the scaling from its own training part only (a unit test checks this). Capital-letter features are log-transformed (`log1p`) then standardised; all other features are standardised. There is no imputation (no missing values) and no encoding (all features are numeric).

Models and grids. A `DummyClassifier` that always predicts ham gives the floor. Logistic regression: penalty L1 or L2, `C` in {0.01, 0.1, 1, 10, 100}, with or without the log transform. k-NN: 5 to 51 neighbours, with uniform or distance weighting. Linear SVM (`SVC` with a linear kernel and probabilities): `C` in {0.01 … 100}. All grids are in `config.yaml`.

Validation protocol. Every choice is made on the training set with stratified 5-fold cross-validation (shuffled, seed 42), using `GridSearchCV` with average precision. The decision threshold is chosen on out-of-fold probabilities (`cross_val_predict`): the lowest threshold whose precision reaches 0.95. The test set is used once, at the end.

Second version. Before looking at the test set, we also decided to train the final model without the nine donor words, with its own threshold chosen the same way, to measure how much the model relies on them. The test set reports both versions; the choice of the final model was made on cross-validation.

5. Results

5.1 Model selection

Configuration (cross-validation, training set)	Best settings	Avg. precision	Std. dev.
Logistic regression, log transform (chosen)	**L1, C = 1**	**0.954**	**0.011**
k-NN, tuned	21 neighbours, distance	0.954	0.005
Linear SVM, tuned	C = 100	0.953	0.012
Logistic regression, no log transform	L1, C = 1	0.943	0.011
Logistic regression, without donor words	L2, C = 0.1	0.938	0.014
k-NN, default settings	5 neighbours	0.921	0.011
Dummy baseline (always ham)	–	0.399	0.000

Table 2. Cross-validated average precision (5 folds). One standard error of the best score is 0.011 / √5 ≈ 0.005.

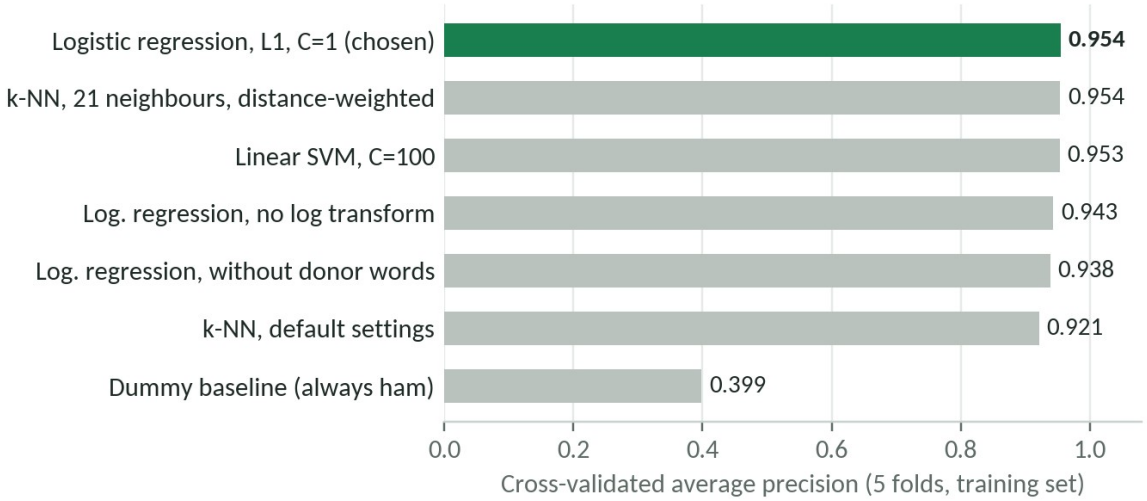


Figure 1. Average precision per configuration.

- **Log transform: kept.** It raises the score from 0.943 to 0.954, about two standard errors.
- **L1 vs L2: L1.** The best L1 and L2 models differ by 0.001, within one standard error; L1 also removes the two redundant features (857, 415).
- **Final model: logistic regression.** After tuning, the three models are within 0.0011 of each other, far below one standard error. Tuning mattered most for k-NN (0.921 → 0.954). Following our rule we keep the simplest model, which also gives probabilities directly and readable coefficients.

5.2 Decision threshold and test set

The out-of-fold precision reaches 0.95 from a threshold of 0.782 (0.82 for the version without donor words), saved in `config.yaml` so the command-line tool uses it too. The tuned hyperparameters were chosen on the same training data, so this estimate is slightly optimistic; the test set checks it.

Test set, 842 emails	Final model (0.782)	Without donor words (0.82)	Meaning
Precision (spam)	**0.950**	**0.952**	**share of flagged emails that are spam**
Recall (spam)	0.795	0.711	share of spam caught
F1 / ROC-AUC	0.866 / 0.977	0.814 / 0.964	ranking at every threshold
Average precision	0.959	0.940	
Legitimate emails flagged	14 of 506	12 of 506	the cost GrandPied watches
Spam missed	69 of 336	97 of 336	deleted by hand

Table 3. The one-time evaluation on the test set, for both versions.

The test precision of the final model matches the 0.95 targeted on out-of-fold predictions, which is consistent with the threshold choice. The test set is small, though: with 281 flagged emails, the precision could lie anywhere between 0.918 and 0.970 (95 % Wilson interval). Without the donor words, the model keeps its precision by flagging fewer emails, so more spam gets through: this is roughly what those words were worth on Spambase.

5.3 What the model learned

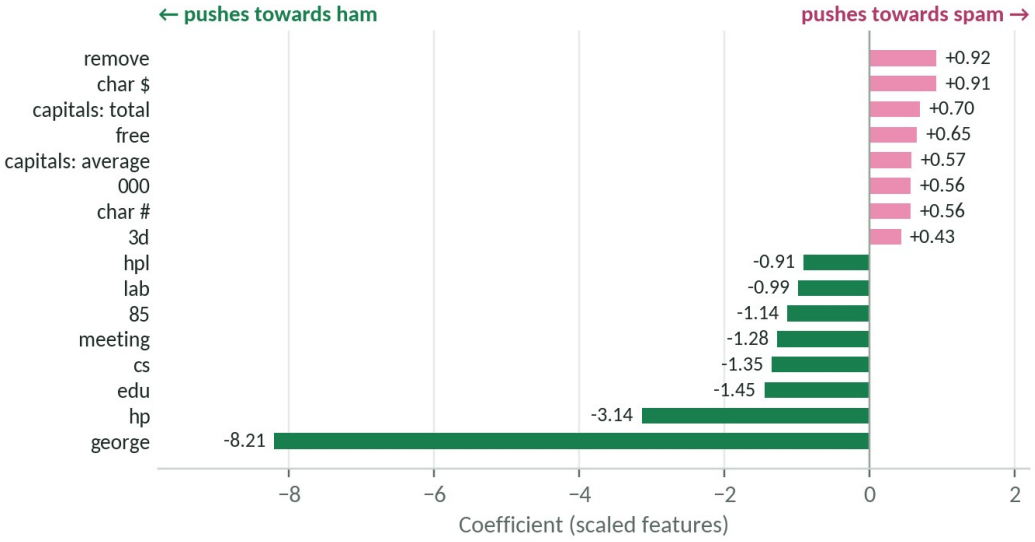


Figure 2. The eight strongest weights in each direction for the final model (features are standardised, so sizes are comparable).

Towards spam, the strongest signals are `remove` ("remove me from this list"), the `$` sign, the number of capital letters and `free`, which match common sense. Towards ham, the donor's own words dominate: `george` (−8.2), then `hp`, `edu`, `cs`, `meeting` and `lab`. Removing the nine donor words lowers the cross-validated average precision from 0.954 to 0.938 (about three standard errors). Cross-validation cannot tell that this gain would not carry over to GrandPied, because every fold comes from the same donor. The second version shows the difference in practice: a short spam that starts with "Hi George," scores 0 % with the final model and 100 % without the donor words.

Published results on Spambase also use all 57 features: the UCI documentation reports about 7 % error, Hastie, Tibshirani and Friedman's textbook uses the same data with `george` and `hp` among its signals, and Shalizi's course at Carnegie Mellon fits a logistic regression on all 57. None of them removes the donor words, which the UCI documentation itself says a general-purpose filter would need. Our final model makes 9.9 % errors at a threshold chosen for precision, rather than the error rate those results optimise.

6. Software

The code is an installable Python package (`pip install -e .`) with every setting in `config.yaml` and exact dependency versions in `pyproject.toml`.

- **Classes.** `DatasetLoader` (`data.py`) downloads Spambase, removes duplicates and saves the stratified split; `Trainer` (`models.py`) builds the pipeline, tunes a model with cross-validation, evaluates, saves and loads it.
- **Command-line tool.** `spamjam train` and `spamjam evaluate` handle both versions; `spamjam predict --input new_case.json` predicts one email from its features, `--text` from the email itself (or a whole folder of emails), and `spamjam features` turns labelled emails into a dataset, so GrandPied could retrain on its own mail.
- **Streamlit page.** Paste an email or edit its features and see the verdict, the probability and the features that pushed it, with a switch for the second version.
- **Quality.** 67 pytest tests (data, no leakage, training, threshold, prediction, text features, command line, Streamlit page), which pass on a fresh clone before any training, ruff for style, names and docstrings, and a fresh-install check.

7. Limitations, risks and recommendations

- **One donor, old data.** All legitimate emails come from one person around 1999, so donor words inflate the scores and modern spam may be missed.

- **English only, and GrandPied's emails look like spam.** The 48 word features are English: a French version of a spam that scores 100 % in English scores 38 %. And everyday marketing words (`free`, `our`, `business`) push towards spam: an ordinary email about a client order scores 96 %.
- **The 0.95 precision depends on the share of spam.** The model flags 2.8 % of legitimate emails and catches 79.5 % of spam whatever the mix, but precision falls when spam is rarer: 0.95 with 40 % spam (Spambase), 0.88 with 20 %, 0.76 with 10 %. The threshold must be chosen again on GrandPied's own mail.
- **Risks for real users.** Errors: even at 0.95 precision some client emails are flagged, so they must go to a quarantine folder, not be deleted. Fairness: emails unlike the donor's (French, marketing) are judged more harshly. Privacy: the model reads every email, which requires informing employees and complying with the GDPR.
- **Recommendations.** Real spam filters are personal: they learn which words matter from each user's own sorted emails (the open-source filter SpamAssassin, and Gmail). For GrandPied, that means retraining on its own labelled emails, per team or per employee, choosing the threshold again, keeping a human in the loop and monitoring how many legitimate emails get flagged.

The full list of limitations, with the numbers behind each point, is in Appendix A.

References

Hopkins, M., Reeber, E., Forman, G. and Suermondt, J. (1999). Spambase [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C53G6X

Hastie, T., Tibshirani, R. and Friedman, J. (2009). The Elements of Statistical Learning, 2nd ed. Springer.

Shalizi, C. Data Mining (36-462), homework 2. Carnegie Mellon University. https://www.stat.cmu.edu/~cshalizi/dm/22/hw/02/hw-02.pdf

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Apache SpamAssassin, sa-learn manual; Google Workspace, An overview of Gmail's spam filters.

Code: github.com/alajemba-vik/spamjam. Use of AI tools is described in AI_USAGE.md.

Appendix A. Limitations

This appendix lists what SpamJam can't do, with the numbers behind each point. Unless said otherwise, the numbers are for the final model on the test set (842 emails, threshold 0.782) and are computed in `notebooks/analysis.ipynb`; the same text is in `docs/limitations.md` in the repository.

1. The real emails all come from one person

All the legitimate emails in Spambase come from the donor's own mailbox (George, at HP Labs, around 1999). Words that describe him are among the strongest signals the model uses: `george` has the largest weight by far (about −8.2, towards a real email), followed by `hp`, `edu`, `cs`, `meeting`, `85` and `lab`. In the training set, `hp` appears in about 40 % of real emails and 3 % of spam.

These words help on Spambase only because they identify the donor's emails. At GrandPied they mean nothing, so our scores are likely optimistic. The UCI documentation itself says a general filter would have to "blind" these words or use real emails from many people.

To measure this, we also train a second version of the model without them (`spamjam predict --without-donor-words`, and the switch on the Streamlit page):

	All features	Without the donor's words
CV average precision	0.954	0.938
Threshold (for 0.95 precision)	0.782	0.82
Test precision	0.950	0.952
Test recall	0.795	0.711
Real emails flagged (of 506)	14	12
Spam missed (of 336)	69	97

Without the donor's words, the model keeps the same precision but catches less spam: that is roughly what the donor's words were worth on Spambase. It is still optimistic for GrandPied, because the other limitations below stay the same.

The published results we compare with (UCI; Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, Table 1.1; Shalizi at CMU) also train on all 57 features, so the comparison is fair, but none of them removes these words either.

2. English only

The 48 word features are all English words (`free`, `money`, `remove`, `business`, ...). A French email matches none of them, so the model only has the punctuation (`!`, `$`, `(`, `;`, `[`, `#`) and the capital letters to go on.

Email	Spam probability	Verdict
Get FREE money today!!! Click here to claim your $500.	100 %	quarantine
Gagnez de l'argent GRATUIT aujourd'hui !!! Cliquez ici pour recevoir vos 500 euros.	38 %	inbox
Gagnez de l'argent gratuit aujourd'hui. Cliquez ici pour recevoir vos 500 euros.	4 %	inbox

French spam almost never reaches the threshold (78.2 %). French real emails aren't flagged either, so no client email is lost, but the filter does almost nothing for French mail. GrandPied would need French words, which means new features and a model trained on its own French emails.

3. "95 % precision" only holds for Spambase's share of spam

On the test set, 95 % of the emails SpamJam flags really are spam. That number depends on how much of the mail is spam: about 40 % in Spambase. The model's own error rates don't change with the mix (it flags 2.8 % of

real emails and catches 79.5 % of spam), but with less spam, the few real emails it flags make up a bigger part of what it flags:

Share of spam in the mail	Precision
40 % (Spambase)	95 %
20 %	88 %
10 %	76 %
5 %	60 %

We don't know GrandPied's share of spam, so the threshold would have to be chosen again on GrandPied's own emails to keep 95 %.

4. The test results have a margin of error

The test set is small: 281 emails flagged, 506 real emails. With 95 % confidence (Wilson interval):

- precision is between 91.8 % and 97.0 % (we report 95.0 %);
- the share of real emails flagged is between 1.7 % and 4.6 % (we report 2.8 %, 14 of 506).

The threshold was chosen as the lowest one whose out-of-fold precision reaches 0.95, so it sits right at the target. Getting 0.950 on the test set is consistent with that choice, but on another sample the precision could just as well be a little below 0.95.

5. The model only sees 57 fixed numbers

Spambase gives 57 numbers per email, not the email itself. The model never sees the sender, the links, the headers, the attachments or the order of the words, which real spam filters rely on heavily. It also can't learn new words: a spam about "bitcoin" or "crypto" looks the same to it as an email without those words. Some emails can't be told apart at all: after removing exact copies, three pairs of emails in Spambase have the same 57 numbers but opposite labels, one spam and one real email.

6. It is easy to fool

Since a few words carry a lot of weight, a spammer can push an email towards "real" by adding them, or avoid the 48 words altogether. For example:

Email	Spam probability
Get FREE money today!!! Click here to claim your $500.	100 %
Hi George, Get FREE money today!!! Click here to claim your $500.	0 %

The second version, trained without the donor's words, isn't fooled by this one: it gives the "Hi George" spam 100 %. But a spammer could still use other words that point to a real email. Longer spam with many spam words resists this better: our example `examples/new_email.txt` stays at 100 % with "Hi George," added. Image-only spam, which has almost no text, would also get through.

7. Old data, and a split that ignores time

The emails were collected around 1999. Spam has changed a lot since then (phishing, image-only spam, obfuscated words). The dataset has no dates, so our split is random: the test set isn't "future" emails, while in real use the model would always predict on newer emails than it was trained on. This makes our test score more optimistic than real use.

8. Emails pasted on the Streamlit page are approximate

The page (and `spamjam predict --text`) computes the 57 numbers from a pasted email with the formulas in the UCI documentation. The documentation doesn't say whether the headers were counted. They probably were: every one of the 4,601 emails has at least one capital letter (the minimum is 1 in all three capital-letter columns, see `docs/data_dictionary.md`), while a pasted email body may have none, a value the model never saw. So pasted emails get values close to Spambase's, not guaranteed identical.

9. Choices GrandPied would revisit

- The 0.95 precision target is our choice. It isn't derived from what a lost client email actually costs GrandPied compared with a spam that gets through.
- We compared three models (logistic regression, k-NN, linear SVM). Naive Bayes, the classic spam filter, and tree-based models weren't tried.
- The "Pourquoi ?" chart on the Streamlit page shows each feature's share of the decision. Features that appear together (`857` and `415`, `labs` and `telnet`) share the weight between them, so a single bar can understate how much that group matters.

10. GrandPied's own emails look like spam to it

GrandPied is a marketing company, and the words its real emails use every day are the words the model learned from spam: `free` (+0.65), `our` (+0.34), `business` (+0.32), `credit`, `internet`, `your` and `you` all push towards spam. Short emails between two colleagues show it:

Email	Spam probability	Verdict
Hi Claire, can we move tomorrow's call with the client to 3 pm? Thanks, Paul	0 %	inbox
Hi Claire, thanks for your order. Our team will send you the campaign report and the free trial details by email today. Best, Paul	96 %	quarantine
Hi Claire, our new offer for your business: order before Friday and get free delivery on your next campaign. Best, Paul	100 %	quarantine

On Spambase, these words mostly come from spam, because the donor worked in a research lab. At GrandPied, many real emails, including the ones to and from clients, would be flagged. The second version doesn't fix this: it gives the "thanks for your order" email 94 %, above its threshold of 82 %.

Risks of deploying it

- **Errors.** Even at 95 % precision, real emails get flagged (14 of 506 on the test set), and at GrandPied far more (see point 10). Each one may be a lost client inquiry, so flagged emails should go to a quarantine folder that people check, never be deleted.
- **Bias.** The model learned what a normal email looks like from one person's mailbox. Emails that don't look like his (French, marketing, written by people outside his field) are judged more harshly, while a spam that mentions "george" or "hp" gets through more easily.
- **Privacy.** The model reads the content of every email (words, characters, capital letters). Using it at GrandPied requires informing employees, clear rules on who can see quarantined emails, and compliance with privacy law (GDPR).

What it means for GrandPied

Before real use, GrandPied would need to retrain the model on its own labelled emails (French and English, marketing included; `spamjam features` turns a folder of emails into a dataset, see the README), choose the threshold again on its own share of spam, send flagged emails to quarantine rather than delete them, and keep an eye on how many real emails get flagged. If clients say their emails went unanswered, the threshold should be raised so that fewer emails are flagged.

Real spam filters go further: they are personal. SpamAssassin, a widely used open-source spam filter that companies run on their own mail servers, can keep a database per user, trained on that user's own sorted mail, and Gmail learns from what each user marks as spam or not spam. Nobody picks the words by hand: the filter takes every word from the emails and learns which ones matter. That is how a word like `george` ends up meaning "real email" for one person. Spambase can't do this, because its 48 words were chosen once, in 1999. For GrandPied, the industry-standard version of SpamJam would learn its words from GrandPied's emails (company and client names, French words) and be trained per team or per employee. With our 57 fixed features, the closest step is retraining on GrandPied's labelled emails with `spamjam features`; learning the words from the emails themselves would mean changing the features, which we haven't built.